

Few words on NodeRED

Gilles Menez - UCA - Dépt. Informatique

March 24, 2026

1 Node-RED

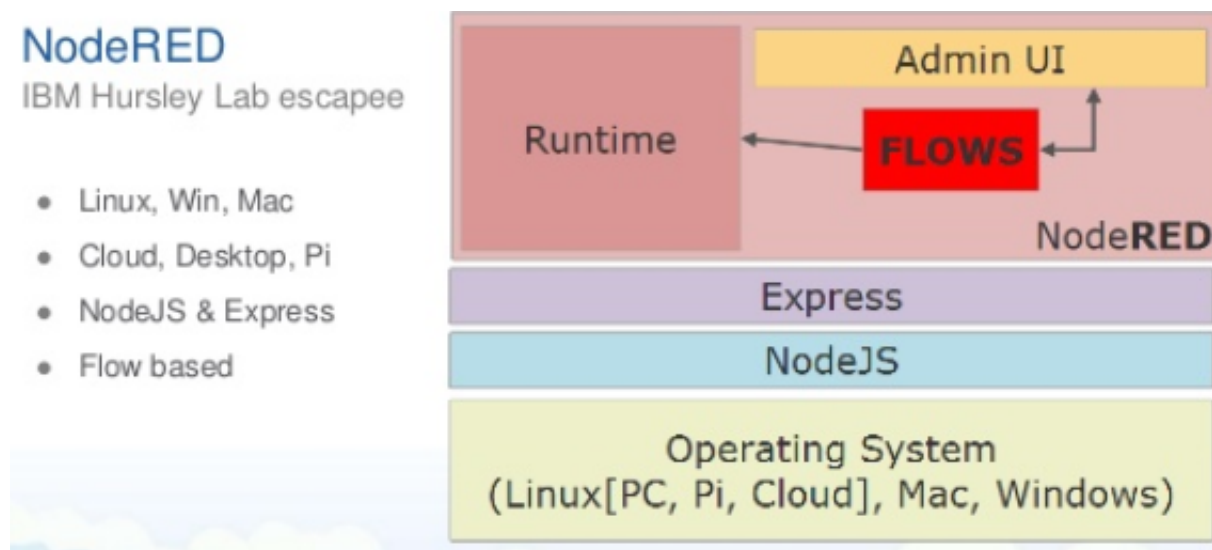
Node-Red (<https://nodered.org/>) est une proposition d'IBM permettant

- ① de **lier aisément des sources de données à des composants** de traitement, locaux ou distants,
- ② et de **créer des chaînes de valeurs** en quelques clics dans un environnement Web.

Un outil "NoCode" ... ou presque !

Techniquement, Node-RED est écrit en Javascript et repose au runtime sur node.js.

- Du coup, NR peut s'exécuter sur beaucoup de plateformes/OS.

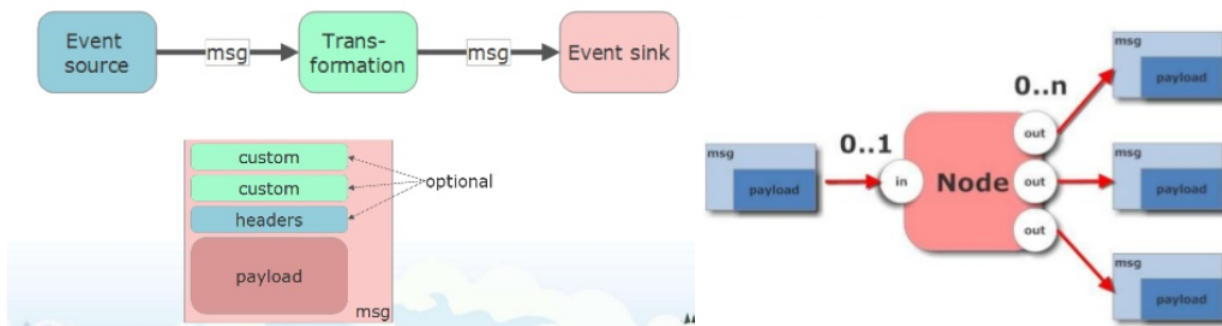


<https://www.slideshare.net/AmitChaudhary112/salesforce-apex-hours-node-red-for-salesforce>

Node-RED utilise **une approche de programmation visuelle** qui permet aux développeurs de connecter des blocs de codes ensembles.

NodeRED

Messages flow from sources to sinks



Ces blocs de codes correspondent à des **noeuds**.

Ils contiennent les traitements qui, une fois connectés par des arcs, constituent un **"flow"** de **messages**.

- Ce "flow" formant ainsi un traitement qui intègre d'autres traitements.

Les flows créés sont stockés/sérialisés au format JSON et il existe d'ailleurs une bibliothèque de "flows" en ligne :

<https://flows.nodered.org>.

1.1 "Event-Driven"

cf <https://nodered.org/docs/user-guide/messages>

et <https://flowfuse.com/node-red/getting-started/node-red-messages/>

Node-RED est piloté par les événements ("event-driven"), ce qui signifie

1. qu'un émetteur d'événements déclenche des événements
2. et qu'un auditeur y réagit.

Dans ce contexte, les **noeuds** de Node-RED **agissent à la fois en tant qu'émetteurs et auditeurs d'événements**, les **messages** servant de support à la communication de ces événements.

1.2 Messages "Node-RED"

Les **messages** sont des "paquets" de données qui voyagent entre les nœuds de votre flux.

Chaque message transporte des données que les nœuds traitent (analyse/modification/...) puis transmettent au nœud suivant dans la séquence.

- Une mauvaise gestion des messages peut entraîner des bogues, tels que l'écrasement de données, des boucles infinies ou des plantages.

Les messages Node-RED sont des objets JavaScript :

```
{
  obj: "kitchen",
  temp: 24,
  verf: true
}
```

Par défaut, ces messages sont des variables nommées "msg".

Ce "msg" contient donc 3 **propriétés** :

1. obj,
2. temp
3. et verf.

1.2.1 Anatomie des messages Node-RED

Les messages Node-RED se composent de plusieurs propriétés essentielles pour le traitement des données et la communication entre les nœuds :

- **msg._msgid** : Un identifiant unique pour le message, automatiquement attribuée par Node-RED.
Il permet de suivre et de déboguer les messages dans le flux.
Cette propriété est toujours présente (car automatiquement attribuée par Node-RED).
- **msg.payload** : Il s'agit des données principales du message.
Il contient les principales informations que les nœuds traitent et sur lesquelles ils agissent, telles que les relevés de capteurs ou les données d'entrée de l'utilisateur.
- **msg.topic** : Voilà un exemple de propriété facultative souvent utilisée pour classer ou identifier le message.
Elle peut être utile pour acheminer ou filtrer les messages en fonction de leur contexte.

Il s'agit là des propriétés les plus courantes (mais pas nécessairement présentes pour topic et payload) que vous rencontrerez dans Node-RED, mais d'autres propriétés peuvent être ajoutées si nécessaire.

Les valeurs des propriétés peuvent être n'importe quel type Javascript et notamment :

- Boolean - true, false
- Number - eg 0, 123.4

- String - "hello"
- Array - [1,2,3,4]
- Object - "a": 1, "b": 2
- Null

1.2.2 "Jouer" avec les messages

Vous trouverez dans les références données au début de la section différentes méthodes pour manipuler les messages NR.

- "Change node"
- "Function node"
- Ajouter des propriétés,
- Les messages et JSON,
- ...

Surtout lire la partie "Common Mistakes to Avoid with Node-RED Messages" !

1.3 Noeuds Node-Red

Trois catégories de noeuds :

1. Les **noeuds d'"input"** permettent d'**injecter des données** dans le flot.

Ces données peuvent provenir d'un port série (celui qui sort de l'ESP ?), d'une requête HTTP, d'un message MQTT, d'un clic souris (noeud inject), ...

Ils possèdent un carré gris sur leur côté gauche.

2. Les **noeuds d'"output"** permettent d'**envoyer des données dans le monde "réel" ou vers le debug**.

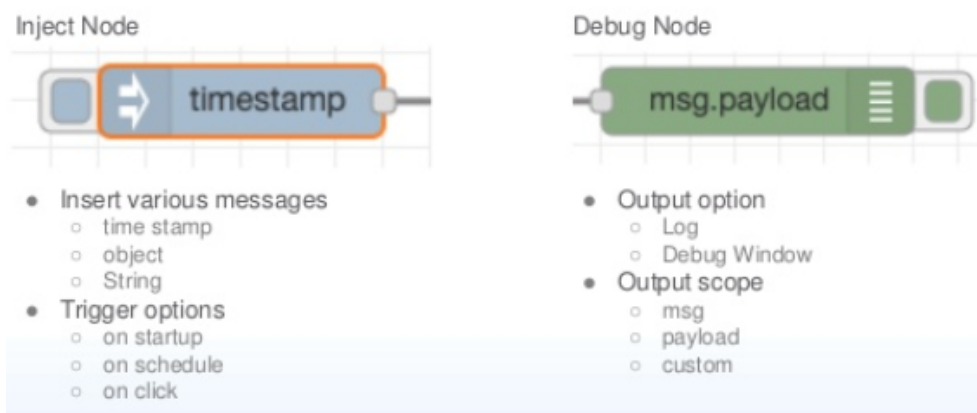
Là encore on pourra utiliser de nombreux protocoles : HTTP, Websocket, ...

Ils possèdent un carré vert sur leur côté droit.

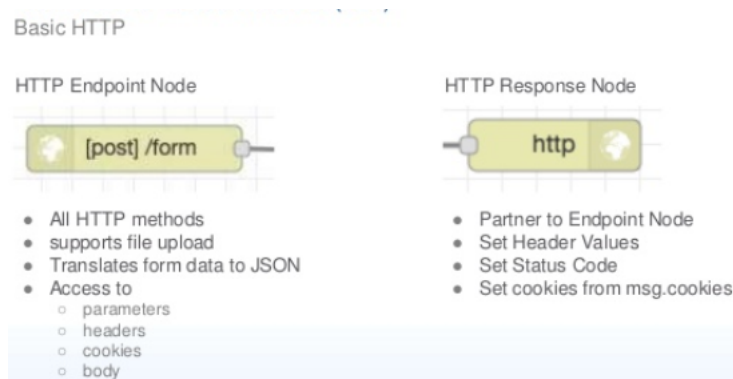
3. Enfin les **noeuds de "traitements"** seront utilisés pour **traiter les données** dans NodeRed.

Par exemple, le noeud "function" permet de le remplir avec du code JS qui sera appliqué à chaque message qui le traverse.

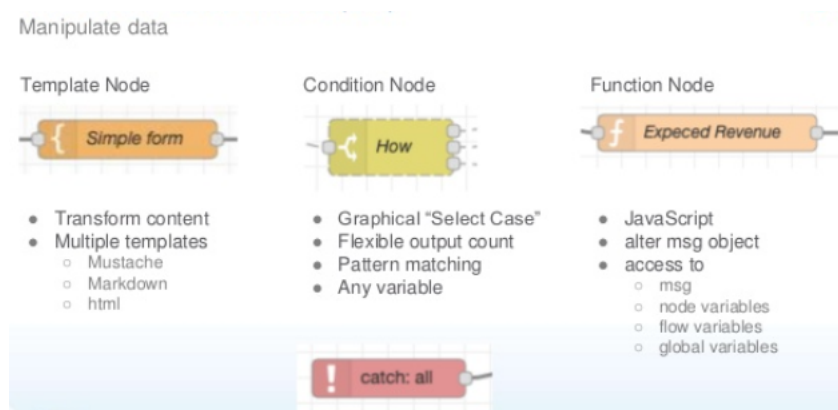
1.3.1 Quelques exemples de noeuds :



- Le noeud "inject" permet d'**injecter un message** dans un flow.



- Le noeud "debug" permet de **visualiser dans la console** le contenu d'un message.
- D'autres noeuds permettent de générer des requêtes et des réponses HTTP.



- D'autres noeuds encore permettent de manipuler les messages et les données qu'ils transportent.
- etc !

2 Installation de Node-RED

Les prérequis sont `node` et `npm` ("son" gestionnaire de paquets) .

Pour installer nodejs sur mon Ubuntu 22.04 en utilisant un paquetage :

<https://github.com/nodesource/distributions#ubuntu-versions>

Même sous Linux cela change souvent mais cela donne à peu près :

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://deb.nodesource.com/gpgkey/nodesource-repo.gpg.key | sudo gpg --dearmor -o /etc/apt/keyrings/nodesource.gpg
echo "deb [signed-by=/etc/apt/keyrings/nodesource.gpg] https://deb.nodesource.com/node_18.x nodistro main" | \
sudo tee /etc/apt/sources.list.d/nodesource.list
sudo apt-get update
sudo apt-get install nodejs -y
node --version
npm --version
```

Ensuite Node-Red s'installe comme un package :

<https://nodered.org/docs/getting-started/local>

Donc,

```
sudo npm install -g --unsafe-perm node-red
```

2.1 Node-RED sous docker

C'est aussi une solution.

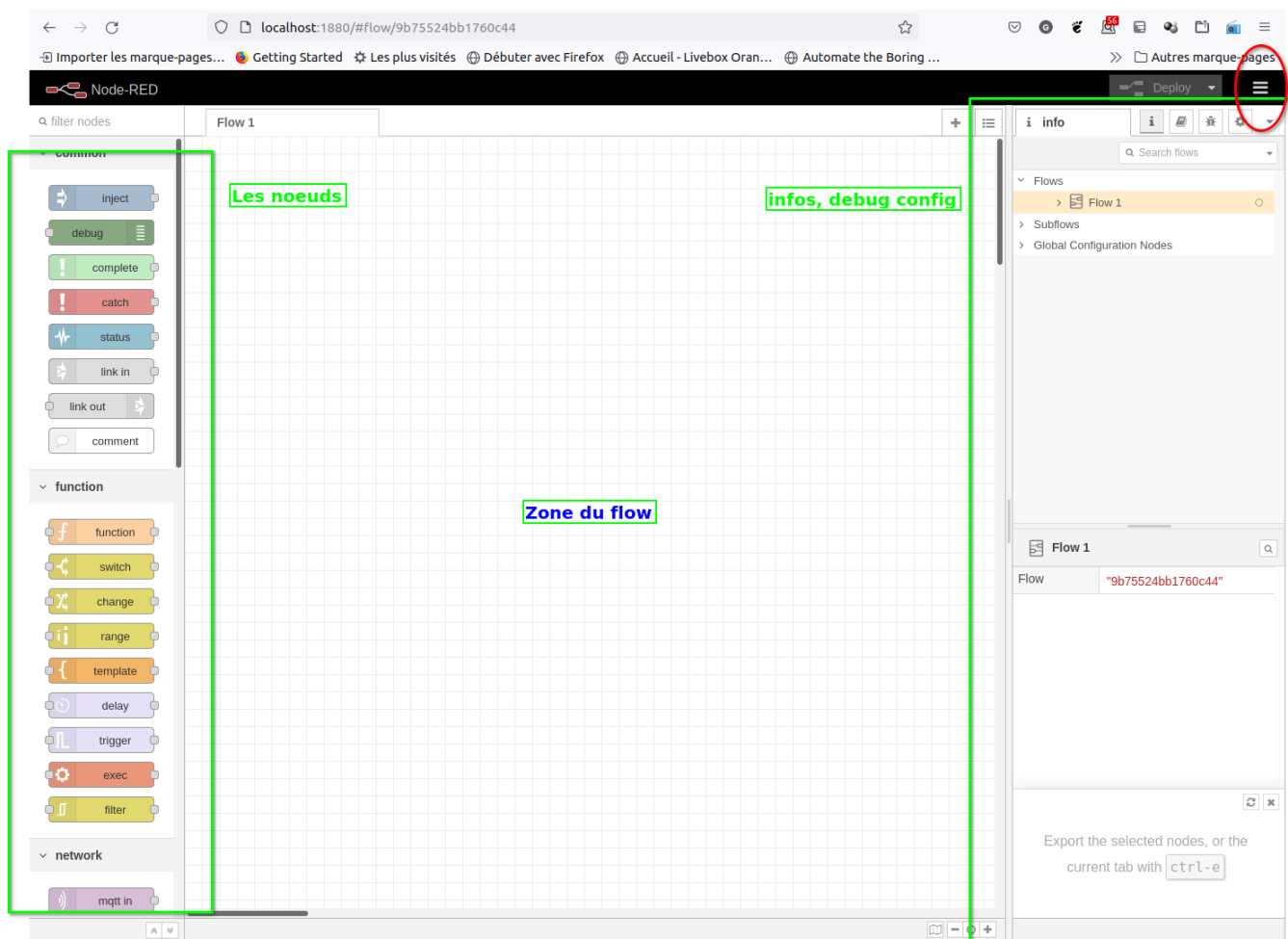
- Il faudra configurer les E/S usb, réseaux, ...

2.2 Execution

Dans un terminal, vous exécutez "node-red" et dans un navigateur vous pouvez alors accéder à l'interface grâce à l'URL :

`http://localhost:1880`

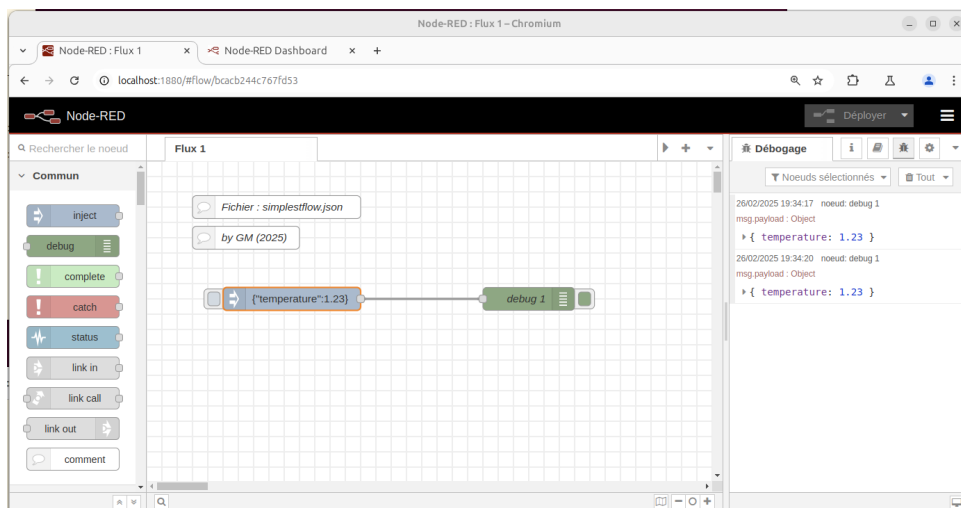
Sur l'écran, vous obtenez



1. A gauche les noeuds,
2. au centre le flow (vide pour l'instant)
3. et à droite infos, debug, config ... sélectionnables par le "hamburger".

3 Deux exemples ...

3.1 Un exemple ... pour voir le debug

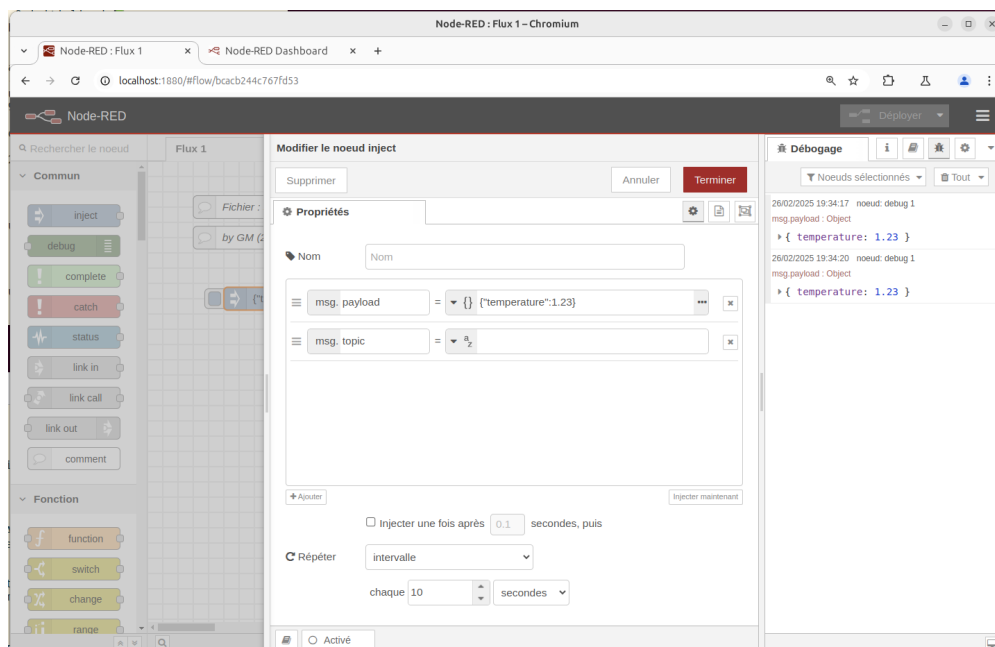


Sur ce flot, on a placé deux noeuds :

1. un noeud d'injection :

Ce noeud permet de "fabriquer" un texte (ici en json) qui sera la payload/charge du message qui sera injecté périodiquement dans le flow du fait du choix de la répétition.

En double cliquant dessus on peut le configurer :



Vous remarquez qu'un message contient une information/charge/payload.

Il est aussi caractérisé par un "topic" qui peut permettre de distinguer des "streams" dans un même flot.

- <https://nodered.org/docs/user-guide/messages>
- <https://nodered.org/docs/developing-flows/message-design>

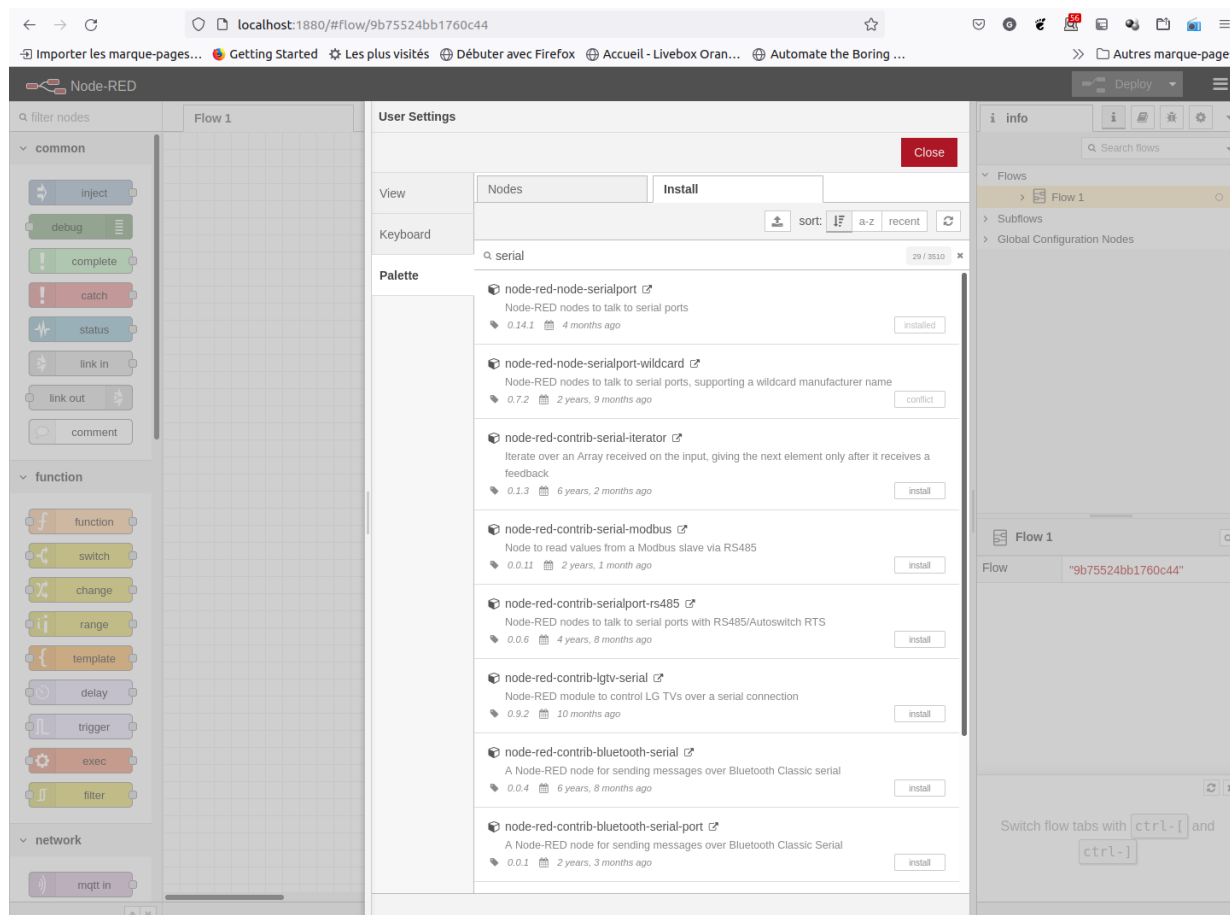
2. un noeud debug :

Ce noeud permet de voir les message sur la console à droite de la page Web.

3.2 Un exemple ... pour "lire" le port série USB ... et donc l'ESP !

3.2.1 Enrichir Node-Red

Pour commencer, rendez-vous sur votre interface Node-RED et accédez (menu coin haut droit) au gestionnaire de Palette ("Manage Palette") pour installer le package officiel : `node-red-node-serialport`



Cette installation introduit 4 nouveaux noeuds dans Node-Red :

<https://flows.nodered.org/node/node-red-node-serialport>

3.2.2 Composer votre flow

Ensuite, vous placez dans le flow différents composants :

- Un noeud de type **"input"** (carré gris à droite) qui va permettre d'injecter des données dans le flow.

Vous choisissez le **"serial in"** dans la catégorie/palette **"network"** qui, comme son nom l'indique, permet d'utiliser le port série, plus précisément ce qui va sortir du port série et que l'on va injecter dans le flow.

Il faut paramétrer ce noeud pour préciser le port série physique de la machine qu'il représente :

The screenshot shows the Node-RED web interface in a browser window at localhost:1880. The left sidebar contains a palette of nodes categorized into 'filter', 'network', and 'sequence'. The 'network' category is expanded, showing various input and output nodes like 'mqtt in/out', 'http in/out', 'websocket in/out', 'tcp in/out', 'udp in/out', and 'serial in/out'. The 'sequence' category shows 'split', 'join', and 'sort' nodes. The main workspace displays a flow named 'Flow 1' with a 'SerialUSB0' node. The right sidebar shows the 'info' panel with a search bar and a list of flows. The 'Edit serial-port node' dialog is open, showing the following configuration:

- Serial Port:** /dev/ttyUSB0
- Settings:** Baud Rate: 9600, Data Bits: 8, Parity: None, Stop Bits: 1. DTR: auto, RTS: auto, CTS: auto, DSR: auto.
- Input:** Optionally wait for a start character of [], then Split input on the character \n, and deliver ASCII strings.
- Output:** Add character to output messages [].
- Request:** Default response timeout 10000 ms.

A tip box states: "Tip: the 'Split on' character is used to split the input into separate messages. Can accept chars (\$), escape codes (\n), or hex codes (0x03)."

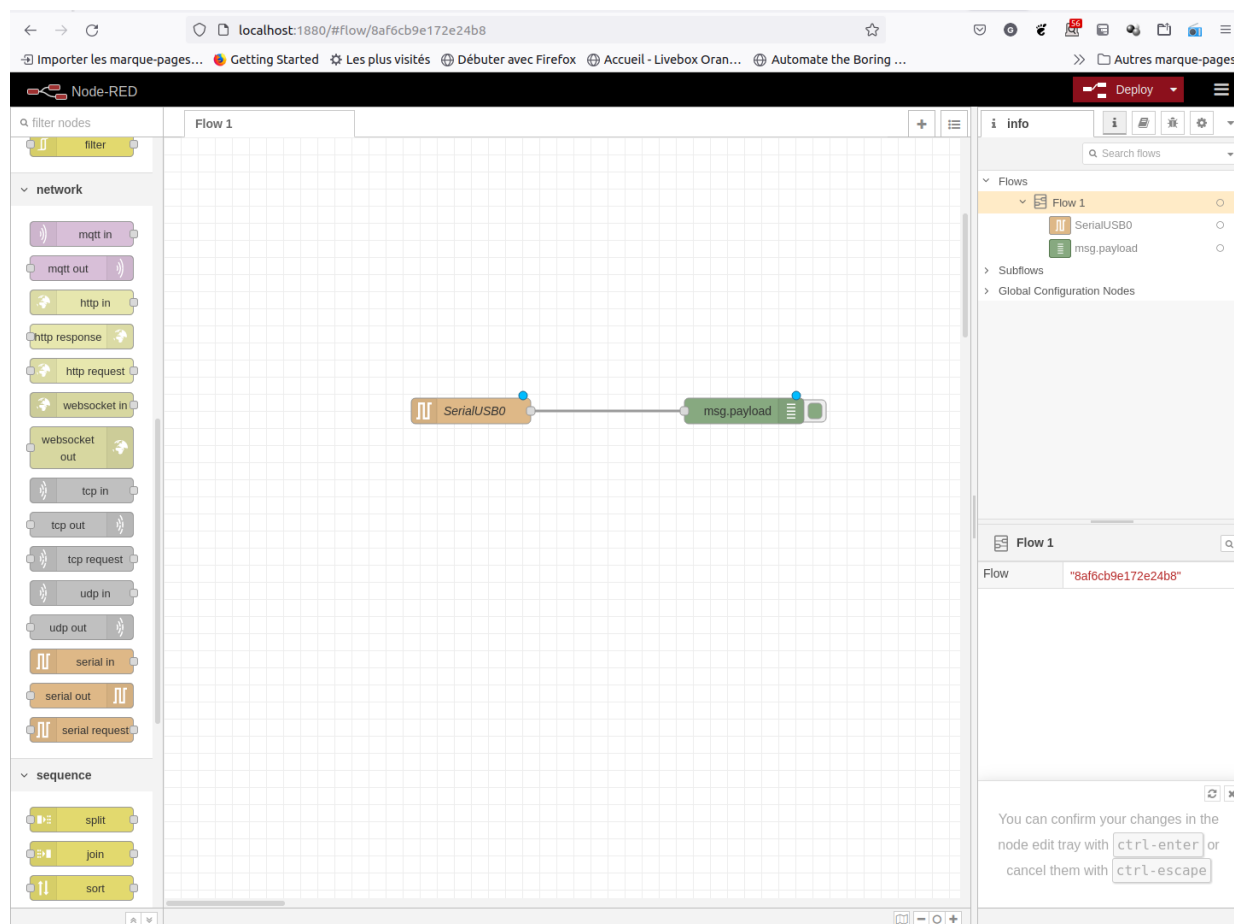
The bottom of the dialog shows the node is enabled and 1 node uses this config. The right sidebar also shows the node details: Node ID "5bb02696.4602", Type serial-port, and a note about managing the palette with alt-p.

- Un noeud de type **"output"** (carré gris à gauche) qui va permettre d'envoyer des données vers le monde réel ou dans notre cas vers une console de debug.

Vous choisissez dans la catégorie/palette "Common" le noeud "debug".

- Et "tirer" un arc entre ces deux noeuds.

Cela donne un truc comme ça :



3.3 Sauver son flot

Avant de fermer/quitter la page, **vous n'oubliez pas de sérialiser/exporter votre flow !**

J'ai exporté mon flow dans "ma" library sous le nom `/home/menez/NodeRed/Data/lib/flows/flow0.json`

```
Terminal
$ cat /home/menez/NodeRed/Data/lib/flows/flow0.json
[{"id":"8af6cb9e172e24b8","type":"tab","label":"Flow 1","disabled":false,"info":""}, {"id":"46aa5571ed472515","type":"serial in","z":"8af6cb9e172e24b8","name":"SerialUSB1","serial":"5bb02696.4602","x":360,"y":800,"wires":[["d46d4546593fa531"]]}, {"id":"d46d4546593fa531","type":"debug","z":"8af6cb9e172e24b8","name":"","active":true,"tosidebar":true,"console":false,"tostatus":false,"complete":"payload","targetType":"msg","x":680,"y":800,"wires":[]}, {"id":"5bb02696.4602","type":"serial-port","serialport":"/dev/ttyUSB1","serialbaud":"9600","databits":"8","parity":"none","stopbits":"1","waitfor":"","dtr":"none","rts":"none","cts":"none","dsr":"none","newline":"\\n","bin":"false","out":"char","addchar":"","responsetimeout":"10000"}]
```

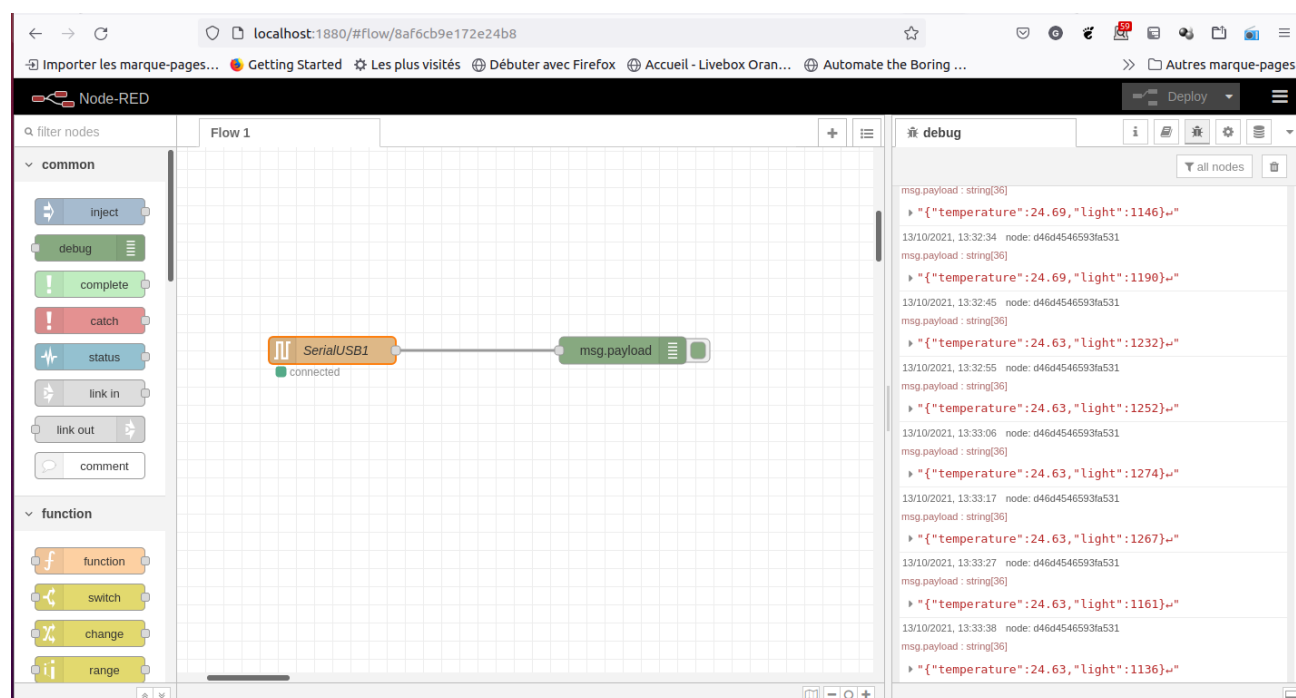
Vous pouvez aussi trouver des flows sous `~/node-red/lib/flows/`

3.3.1 Déployer le flow

Lorsque le flow est crée, on le déploie (cf bouton rouge en haut à droite).

<http://noderedguide.com/node-red-lecture-2-building-your-first-flows-15/> :

- "Click the deploy button in the Node-RED window (top right). You'll see a pop-up saying the flow has been successfully deployed."
- "You will also notice that the blue dots on the nodes disappear, indicating there are no un-deployed changes."



Une fois les noeuds du flow correctement déployés, vous pouvez (théoriquement) visualiser dans le debug (petit icône en haut à droite) les chaînes de caractères qui arrivent sur le port série !

3.3.2 Les informations du Serial port

Pour information, le sketch ESP (qui produit l'information sur le port série) associé à ce flow est le suivant :

```

1  #include "sensors.h"
2  #include "OneWire.h"
3  #include "DallasTemperature.h"
4
5  /* ---- Set timer ---- */
6  unsigned long loop_period = 10L * 1000; /* => 10000ms : 10 s */
7
8  /* ---- LED ---- */
9  const int LEDpin = 19; // LED will use GPIO pin 19
10 // Ces variables permettent d'avoir une representation
11 // interne au programme du statut "electrique" de l'objet.
12 // Car on ne peut pas interroger une GPIO pour lui demander !
13 String LEDState = "off";
14
15 /* ---- Light ----*/
16 const int LightPin = A5; // Read analog input on ADC1_CHANNEL_5 (GPIO 33)
17
18 /* ---- TEMP ---- */
19 OneWire oneWire(23); // Pour utiliser une entite oneWire sur le port 23
20 DallasTemperature tempSensor(&oneWire) ; // Cette entite est utilisee
21                                     // par le capteur de
22                                     // temperature
23
24 /*---- Arduino IDE paradigm : setup+loop -----*/
25 void setup(){
26     Serial.begin(9600);
27     while (!Serial); // wait for a serial connection. Needed for native USB port only
28
29     // Initialize the LED
30     setup_led(LEDpin, OUTPUT, LOW);
31
32     // Init temperature sensor
33     tempSensor.begin();
34 }
35 void loop(){
36     String t = get_temperature(tempSensor);
37     String l = get_light(LightPin);
38
39     //affichage des données sous format JSON ... a l'arrache !
40     Serial.print("{");
41     Serial.print("\"temperature\":");
42     Serial.print(t);
43     Serial.print("\",\"light\":");
44     Serial.print(l);
45     Serial.println("\}");
46
47     //print_topic("temperature",t);
48     //print_topic("light",l);
49     delay(loop_period);
50 }

```

4 Dashboard with Node-Red

L'IoT est une science des données, et il est logique qu'elle soit associée à leurs représentations.

Le dashboard ou "tableau de bord" en français est un "résumé graphique" d'informations "importantes"/"décisionnelles":



- Les dashboards jouent un rôle majeur dans l'interprétation et la compréhension des phénomènes liés aux évolutions des données remontées par exemple par les capteurs des objets.

4.1 Via Node-Red

Node-Red propose des noeuds qui ont une représentation graphique affichable dans une page Web.

- Ceci permet de construire rapidement (programmation graphique) des tableaux de bord.

Pour voir cette page Web contenant les représentation graphiques des noeuds, on peut passer par les menus de l'outil ou suffixer l'URL de l'outil avec "ui"

<http://localhost:1880/ui>

Remarque :

- Si vous lisez <https://flows.nodered.org/node/node-red-dashboard>, vous verrez que l'on peut modifier le nom de l'url "par défaut" de l'UI et aussi son comportement : cf "Setting"

4.2 Layout

L'organisation des objets graphiques sur la page du tableau de bord (dashboard) correspond au "layout".

Les bibliothèques graphiques des langages "classiques" comme Java par exemple proposent de nombreux types de layout :

<https://docs.oracle.com/javase/tutorial/uiswing/layout/visual.html>

Node-Red fait plus simple et s'appuie sur une grille ("grid") de placement.

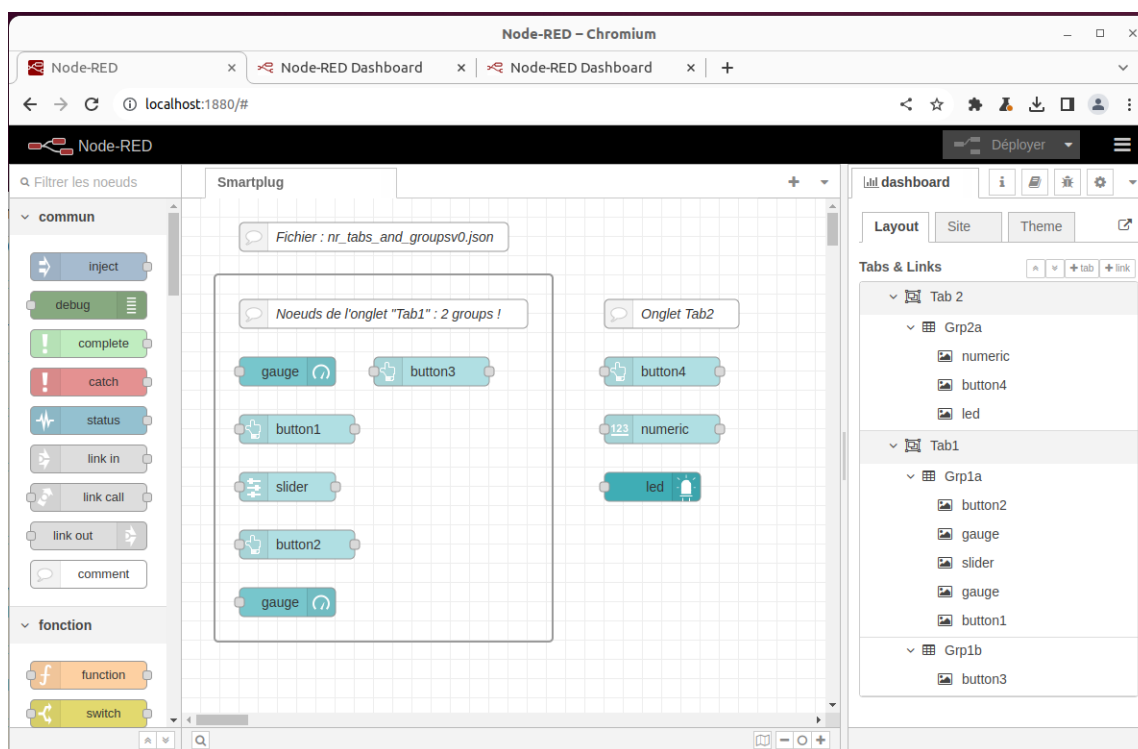
4.3 Tabs and Groups

Un dashboard NR se compose d'onglets (tabs) et de groupes (groups).

- Les **widgets** (représentation graphique des noeuds UI) sont placés dans des groupes,
- et les groupes sont eux mêmes placés dans des onglets.

Au niveau de l'outil, c'est au programmeur

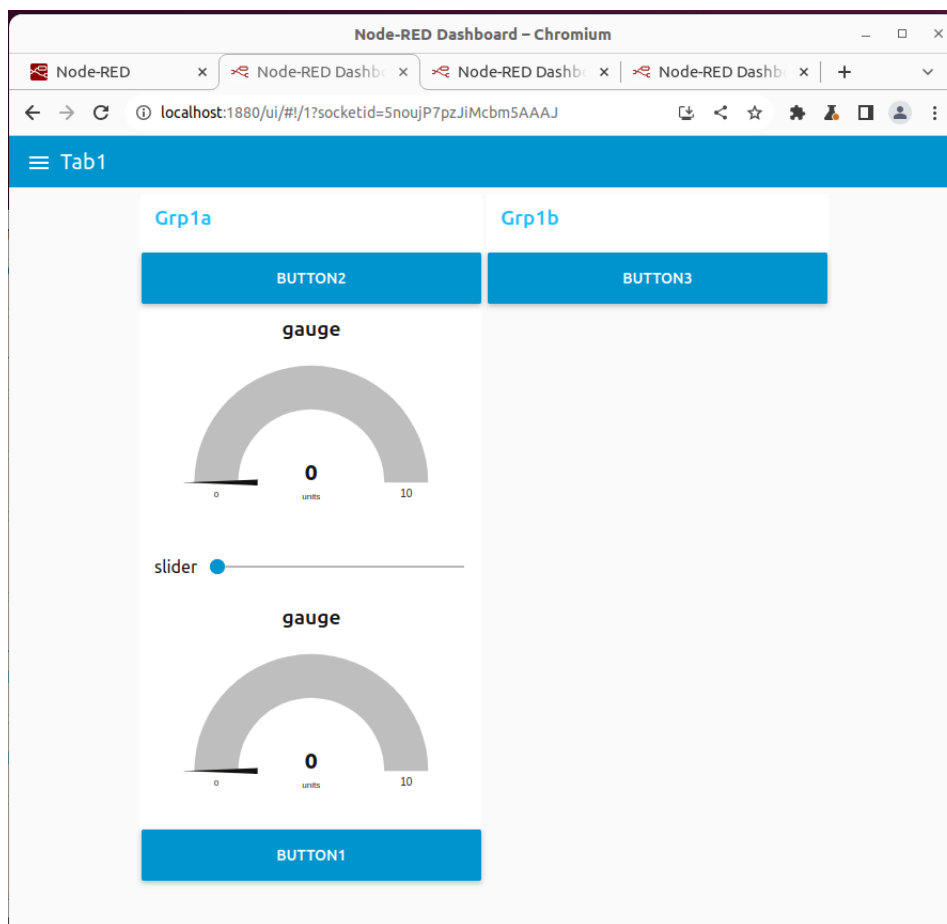
1. de créer "par l'interface" tabs/onglets et groups,
2. puis pour chaque noeud de spécifier (par l'interface toujours) son groupe d'appartenance.



cf <https://flows.nodered.org/node/node-red-dashboard>

Vous remarquez que l'on se situe dans l'onglet "Layout" !

L'interface utilisateur associée à l'onglet "Tab1" est rendue par la figure :



- Chaque élément de groupe a une largeur - par défaut 6 "unités" (une unité a une largeur de 48px par défaut avec un espace de 6px).
- Chaque widget du groupe a également une largeur par défaut, "auto", ce qui signifie qu'il occupera la largeur du groupe dans lequel il se trouve, mais vous pouvez la définir sur un nombre fixe d'unités.

L'algorithme de mise en page du tableau de bord essaye toujours de placer les éléments le plus haut et le plus à gauche possible dans leur conteneur.

- **Cela s'applique à la façon dont les groupes sont positionnés sur la page, ainsi qu'à la façon dont les widgets sont positionnés dans un groupe.**

Dans un groupe de largeur 6, si vous ajoutez six widgets, chacun d'une largeur de 2, ils seront disposés en deux rangées de trois widgets chacune.

Si vous ajoutez deux groupes d'une largeur de 6, tant que la fenêtre de votre navigateur est suffisamment large, ils seront disposés l'un à côté de l'autre. Si vous réduisez la taille du navigateur, le deuxième groupe se déplacera à un moment donné pour se trouver sous le premier, dans une colonne.

Il est conseillé d'utiliser plusieurs groupes si possible, plutôt qu'un seul grand groupe, afin que la page puisse être redimensionnée dynamiquement sur les écrans plus petits.

4.4 Some references

Il ne s'agit pas de devenir le "roi du Dashboard Node-Red" mais si vous en aviez besoin voilà quelques références qui permettent de construire un dashboard en très peu de temps :

- <https://stevesnoderedguide.com/using-control-ui-node-flow-example>
- <https://developer.cisco.com/meraki/meraki-dashboard-api-node-red-node/#!build-a-dashboard/buttons>

For UI/Dashboard in NodeRED :

- <https://nodered.org/docs/user-guide/editor/>
- <https://nodered.org/docs/tutorials/second-flow>
- <https://stevesnoderedguide.com/node-red-dashboard>
- <https://nodered-dashboards.gitbook.io/node-red-dashboards/>
- <https://projetsdiy.fr/node-red-dashboard-gauges-graphiques-notifications-html/>
- <http://noderedguide.com/tutorial-advanced-dashboards-for-node-red-and-cryptocurrency/>
- <https://flows.nodered.org>
- <https://flows.nodered.org/node/node-red-dashboard>
- <https://developer.cisco.com/meraki/meraki-dashboard-api-node-red-node/#!build-a-dashboard/>

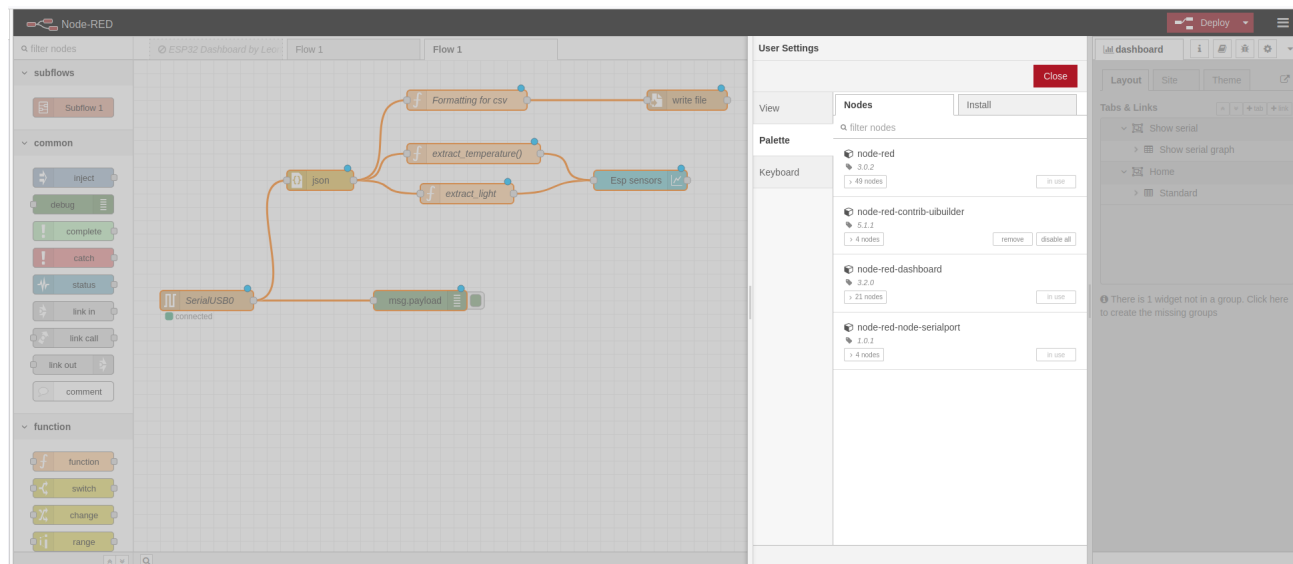
Dynamically created dashboard items :

- <https://discourse.nodered.org/t/dynamically-create-dashboard-items/13841>

4.5 Exemple de dashboard : extraction des données

Il existe des milliers d'outils (en local ou sur le Web) pour composer un dashboard. Si Node-Red n'est pas exclusivement dédié à cela, il intègre tout de même des palettes de noeuds adaptés.

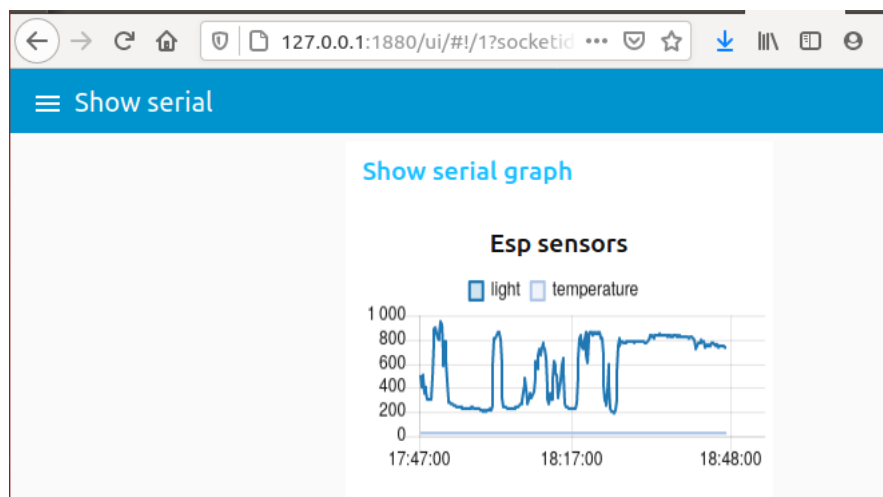
Vous installez les palettes `node-red-dashboard` et `node-red-contrib-ui-led`



21 noeuds sont alors disponibles et leurs documentations sont disponibles là :

<https://flows.nodered.org/node/node-red-dashboard>

Pour l'exemple qui suit nous utilisons le noeud "graph" représenté ci dessous :



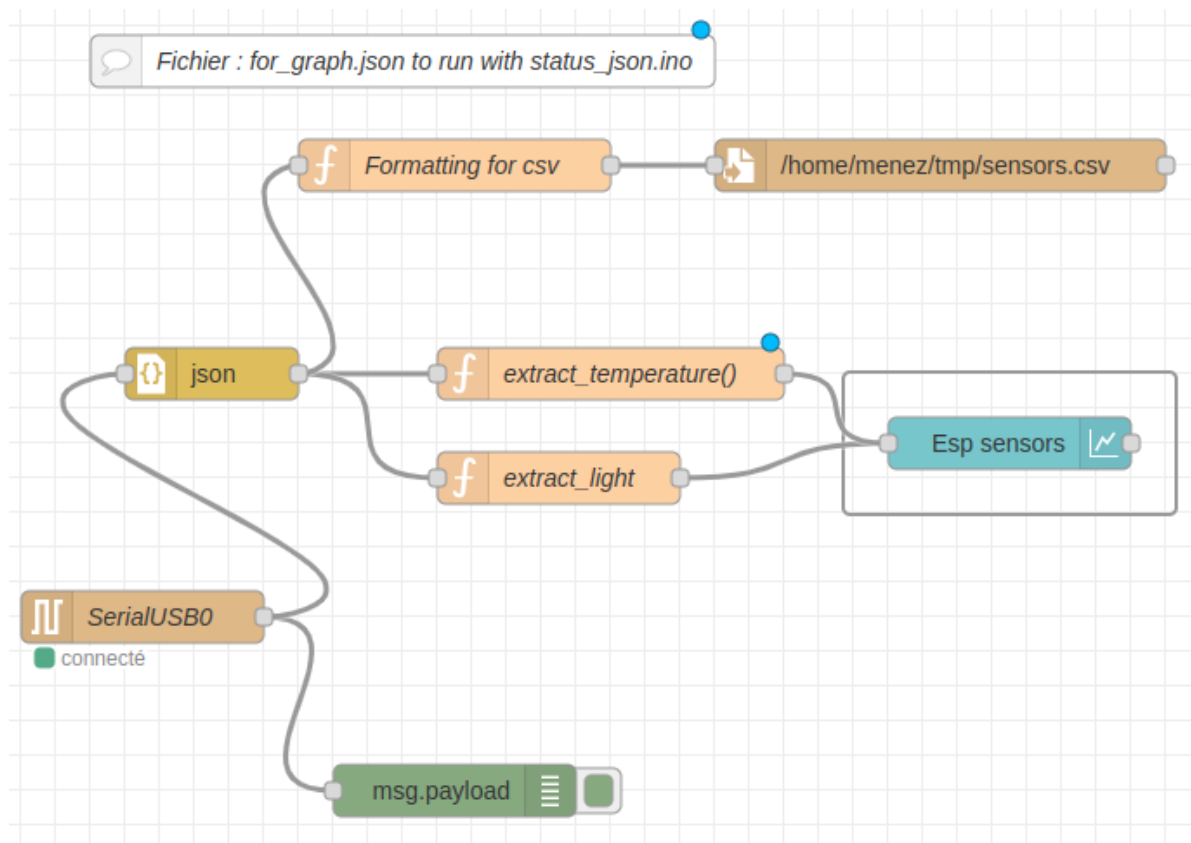
Cet objet Node-RED va constituer implicitement une série à partir des valeurs successives qu'il reçoit.

L'objectif de ce dashboard est d'afficher les données des capteurs de l'ESP sur ce graphe.

- Vous remarquez sur l'url ("127.0.0.1:1880/ui") du dashboard de la page précédente l'utilisation du suffixe "ui" qui correspond bien au dashboard du flow.

4.5.1 On fait ensemble le flot ...

Je vous donne l'image du flow :

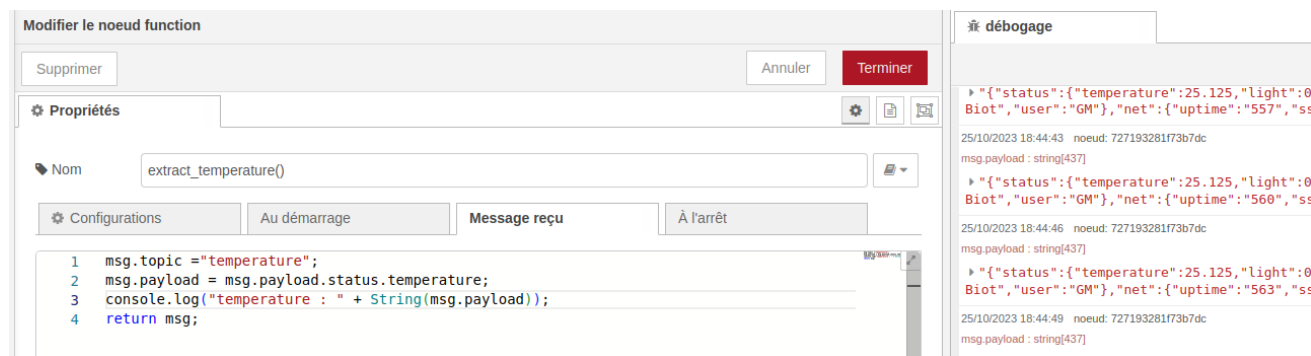


et vous le reconstituez vous même !

Le principe de ce flow est le suivant :

- Le **noeud "SerialUSB0"** récupère à partir du port série une chaîne de caractères JSON produite par un sketch ESP adapté par exemple.
- Le **noeud "json"** convertit cette chaîne en un "objet JSON".
De ce fait, la payload du message sortant est un objet JSON.
- Les **fonctions (extract... ())** accèdent aux champs du dictionnaire JSON de la payload des messages transitant entre les noeuds.
Elles reformulent la payload pour qu'elle soit interprétable par le noeud graph ("Esp sensors").

L'éditeur Javascript apparaît dès que vous double-cliquez sur le noeud !



Ainsi le message sortant envoyé au graphe doit contenir :

1. une payload qui est une nouvelle valeur pour la courbe à tracer.

Cette valeur est obtenue par le champ temperature/light de la payload du message entrant.

La payload entrante est un objet json, et il est logique d'accéder au champ par la notation pointée

"msg.payload.status.temperature"

2. et une référence à la courbe/série si il y en a plusieurs.

C'est le "topic" : Ici "temperature" qui donne la courbe concernée par cette nouvelle valeur.

➤ **Le noeud de formatage** (pour cvs) est un exemple de transformation/exploitation du message et plus précisément de sa payload contenant un objet JSON.

➤ **Le noeud graphe** doit être introduit dans un "group" lui même appartenant à un "tab".

➤ Ceci se fait par l'interface de composition du flow.

Cela participe à la définition du layout de la fenêtre du dashboard que vous obtenez par l'url : "127.0.0.1:1880/ui".

5 TODO : Make YOUR ESP32 Dashboard !

Après avoir essayé ces exemples et acquis une petite maîtrise de Node-RED, proposer votre dashboard permettant de visualiser la charge (payload) envoyée par l'ESP sur le port série (SERIAL).

- Vous faites **au mieux pour représenter le status de l'ESP** sur le dashboard.

La charge sera au format JSON avec un format identique à ce que vous avez codé pour le TODO sur JSON !

Pour vous aider et vous donner des idées ...

- <https://flows.nodered.org/node/node-red-contrib-ui-led/in/590bc13ff3a5f005c7d2189bbb563976>
- <https://www.youtube.com/watch?v=ibe1Ej2Tv7k>
- <https://www.youtube.com/watch?v=C0hCQuj1BM4>